

Comparative Analysis of 16-bit and 32-bit MIPS Single-Cycle Processors: When Algorithmic Efficiency Dominates Architectural Width

Ricardo Herrera-Pineda

*Electrical and Computer Engineering
California State Polytechnic University, Pomona
Pomona, California, USA
rdhernandez@cpp.edu*

Sean Wygant

*Electrical and Computer Engineering
California State Polytechnic University, Pomona
Pomona, California, USA
swygant@cpp.edu*

Daniel Mondragon

*Electrical and Computer Engineering
California State Polytechnic University, Pomona
Pomona, California, USA
damondragon@cpp.edu*

Heba Hafez

*Electrical and Computer Engineering
California State Polytechnic University, Pomona
Pomona, California, USA
hhafez@cpp.edu*

Abstract—We present a comparative analysis of 16-bit and 32-bit MIPS single-cycle processor implementations on FPGA hardware that challenges conventional assumptions about resource efficiency. Through systematic characterization on Nexys A7-100T development boards, we discovered a counterintuitive result: the 16-bit processor consumes 10% more FPGA lookup tables (LUTs) than its 32-bit counterpart (414 vs. 376 LUTs) despite having half the data width. Our investigation reveals that this paradox stems from algorithmic inefficiency in the 16-bit branch calculation logic, which employs 5 operations compared to the 32-bit implementation's 2 operations. This algorithmic complexity penalty overwhelms any resource savings from narrower data paths. Additionally, while the 16-bit processor exhibits 3.5% lower instantaneous power consumption, its 21% longer execution time results in 16% higher total energy consumption. These findings demonstrate that algorithmic optimization dominates architectural width considerations in determining hardware efficiency, with significant implications for educational processor design and embedded systems implementation.

Index Terms—MIPS processor, FPGA implementation, single-cycle architecture, resource utilization, algorithmic efficiency, hardware optimization, comparative analysis

I. INTRODUCTION

The MIPS (Microprocessor without Interlocked Pipelined Stages) architecture has served as a foundational teaching tool in computer engineering education for decades. Its clean instruction set architecture (ISA) and straightforward implementation make it ideal for introducing students to processor design concepts. However, when implementing MIPS processors on FPGA hardware, fundamental questions arise about the relationship between architectural width and resource efficiency.

Conventional wisdom suggests that narrower architectures should consume proportionally fewer hardware resources. A 16-bit processor, operating on half the data width of a 32-

bit processor, might be expected to require approximately half the logic resources. This assumption drives many educational implementations and embedded system design decisions where resource constraints are paramount.

This paper presents a discovery-driven comparative analysis that challenges this conventional assumption. We deployed and characterized two MIPS single-cycle processor implementations—a 16-bit and a 32-bit variant—on identical FPGA hardware (Nexys A7-100T development boards). Through comprehensive measurements of resource utilization, power consumption, timing characteristics, and execution performance, we discovered a counterintuitive paradox: the narrower 16-bit processor consumes more combinational logic resources than the wider 32-bit implementation.

Our systematic investigation revealed that this paradox stems from fundamental differences in algorithmic implementation rather than architectural width. Specifically, the 16-bit processor's branch calculation logic employs a 5-operation manual approach to signed arithmetic, while the 32-bit processor leverages hardware-native operations with just 2 operations. This 150% increase in algorithmic complexity generates sufficient additional logic to overwhelm any resource savings from narrower data paths.

A. Research Questions

This study addresses four interconnected research questions:

RQ1: Does data path width directly correlate with hardware resource utilization in FPGA implementations?

RQ2: How do algorithmic design choices impact resource consumption independent of architectural data width?

RQ3: What are the performance and energy implications of different branch calculation strategies in single-cycle processors?

RQ4: Can these findings inform future educational processor implementations and embedded system design decisions?

Our results demonstrate that algorithmic efficiency dominates architectural width considerations, with practical implications for both educational implementations and resource-constrained embedded systems.

II. BACKGROUND

A. MIPS Architecture

The MIPS ISA features a load-store architecture with 32 general-purpose registers, simple addressing modes, and three instruction formats (R-type, I-type, and J-type). In standard 32-bit MIPS, instructions are 32 bits wide with 6-bit opcodes and 5-bit register specifiers. The 16-bit variant studied here uses a compressed format with 16-bit instructions, 3-bit opcodes, and 3-bit register specifiers, providing 8 registers instead of 32.

B. Single-Cycle Implementation

Single-cycle processors execute each instruction in exactly one clock cycle, with distinct stages: instruction fetch, decode, execute, memory access, and write-back occurring sequentially within a single cycle. While this approach simplifies control logic and eliminates pipeline hazards, it requires a long clock period to accommodate the slowest instruction path.

C. FPGA Implementation Considerations

FPGAs implement digital logic using lookup tables (LUTs), which are essentially small memory-based truth tables that can implement any combinational logic function. Flip-flops provide storage elements for sequential logic. Modern FPGAs like the Xilinx Artix-7 (used in the Nexys A7-100T) contain thousands of configurable logic blocks, each consisting of LUTs, flip-flops, and routing resources.

Resource utilization in FPGAs depends on both logic complexity and data width. While wider data paths require more resources for storage (flip-flops) and data movement, combinational logic complexity is primarily determined by the operations performed rather than operand width. This distinction proves critical to understanding our results.

III. METHODOLOGY

Our research methodology consisted of five systematic phases:

A. Phase 1: Deployment

We deployed two MIPS single-cycle processor implementations on Nexys A7-100T FPGA development boards (Xilinx Artix-7 XC7A100T):

- **16-bit Processor:** Implements 8 instructions (ADD, SUB, AND, OR, SLT, ADDI, SLTI, LW, SW, BEQ, J, JAL, JR) with 8 registers (3-bit addressing) and 16-bit data paths
- **32-bit Processor:** Implements the same instruction subset with 32 registers (5-bit addressing) and 32-bit data paths

Both implementations use identical single-cycle architectures with the same functional units: program counter, instruction memory, register file, ALU, data memory, and control logic. The primary differences lie in data width, register count, and specific implementation details of control logic.

B. Phase 2: Measurement Collection

We collected comprehensive metrics across four categories using Xilinx Vivado Design Suite 2023.2:

Resource Utilization: Synthesis and implementation reports provided exact counts of LUTs, flip-flops, and I/O resources consumed by each design.

Power Consumption: Vivado's power analysis tool generated estimates based on design complexity, clock frequency, and switching activity. We measured at two operating points: (1) 10 kHz clock for observable FPGA demonstration and (2) maximum achievable frequency.

Timing Characteristics: Static timing analysis identified critical paths, setup/hold violations, and maximum operating frequencies. We examined both combinational delays and sequential timing requirements.

Execution Performance: Both processors executed an identical test program: a summation loop computing $\sum_{i=0}^{49} i = 1225$. We instrumented the designs to count total instructions executed and measured actual execution time on hardware.

C. Phase 3: Paradox Discovery

Initial synthesis results revealed an unexpected finding: the 16-bit processor consumed 414 LUTs compared to 376 LUTs for the 32-bit processor—a 10% increase despite having half the data width. This counterintuitive result contradicted the conventional assumption that narrower architectures uniformly reduce resource consumption.

D. Phase 4: Root Cause Investigation

To understand this paradox, we performed systematic code analysis examining each functional unit's implementation. We compared the Verilog source code for both processors, paying particular attention to areas where resource usage might differ unexpectedly. This forensic investigation identified the branch calculation logic as the primary source of inefficiency.

E. Phase 5: Comprehensive Characterization

With the root cause identified, we quantified its impact across all measured metrics. We traced how the algorithmic inefficiency in branch calculation propagated through resource utilization, power consumption, timing, and overall performance. This holistic characterization revealed that the inefficiency affected not just LUT count but also execution speed and energy consumption.

IV. RESULTS

A. Resource Utilization Analysis

Table I presents the complete resource utilization comparison. The results reveal a nuanced picture where the 16-bit processor demonstrates advantages in some metrics while showing significant disadvantages in others.

TABLE I
RESOURCE UTILIZATION COMPARISON

Resource	16-bit	32-bit	Winner
LUTs	414 (0.65%)	376 (0.59%)	32-bit
Flip-Flops	116 (0.09%)	196 (0.15%)	16-bit
I/O Pins	21 (10.0%)	21 (10.0%)	Tie

The 16-bit processor’s 40.8% flip-flop advantage (116 vs. 196) aligns with expectations: storing 16 bits per register requires fewer storage elements than storing 32 bits. However, the 10.1% LUT disadvantage (414 vs. 376) contradicts conventional assumptions about resource scaling with data width.

This paradox demonstrates that combinational logic complexity—implemented in LUTs—does not scale linearly with data width. While sequential storage requirements (flip-flops) follow data width predictably, control and computation logic depends primarily on algorithmic complexity rather than operand size.

B. Performance Analysis

Table II shows execution characteristics for the test program running at a 10 kHz clock frequency on actual FPGA hardware.

TABLE II
PERFORMANCE COMPARISON (10 KHz CLOCK)

Metric	16-bit	32-bit	Difference
Instructions	20,354	16,836	+20.9%
Execution Time	8.07s	6.70s	+20.5%
CPI	1.0	1.0	0%

The 16-bit processor requires 20.9% more instructions to execute the identical summation computation. This overhead stems from two sources: (1) the complex branch calculation requires additional operations, and (2) the loop executes the same number of iterations but with more operations per iteration. Since both are single-cycle implementations with CPI = 1.0, instruction count directly determines execution time.

C. Power and Energy Analysis

Table III presents power and energy measurements, revealing a critical distinction between instantaneous power consumption and total energy consumption.

TABLE III
POWER AND ENERGY COMPARISON (10 KHz CLOCK)

Metric	16-bit	32-bit	Winner
Power (W)	0.833	0.862	16-bit
Time (s)	8.07	6.70	32-bit
Energy (J)	6.72	5.78	32-bit

The 16-bit processor exhibits 3.5% lower instantaneous power consumption (0.833W vs. 0.862W), which might suggest greater efficiency. However, its 20.5% longer execution

time (8.07s vs. 6.70s) results in 16.3% higher total energy consumption (6.72J vs. 5.78J).

This demonstrates a fundamental principle: energy efficiency depends on both power consumption rate and execution time. The relationship $\text{Energy} = \text{Power} \times \text{Time}$ means that a processor consuming more power per unit time can still use less total energy if it completes the task sufficiently faster. In battery-powered applications or systems with energy budgets, total energy consumption matters more than instantaneous power.

D. Timing Analysis

Static timing analysis revealed differences in critical path complexity:

- **16-bit:** 887 timing endpoints analyzed
- **32-bit:** 551 timing endpoints analyzed
- **Difference:** 61% more timing paths in 16-bit design

The increased number of timing endpoints in the 16-bit processor correlates with its higher combinational logic complexity. Each additional operation in the branch calculation creates additional signal paths that must be analyzed and optimized by the synthesis tools. This complexity can impact maximum achievable clock frequency and timing closure difficulty.

V. ROOT CAUSE ANALYSIS

A. Branch Calculation Implementations

Detailed code analysis revealed fundamental differences in how the two processors handle branch target address calculation:

32-bit Implementation (2 operations):

- 1) Shift immediate value left by 2 bits (word alignment)
- 2) Add shifted offset to PC+4

This approach leverages the hardware’s native two’s complement arithmetic. Negative offsets are naturally handled by the adder without explicit sign checks or conditional operations. The synthesis tools optimize this to a single adder with minimal control logic.

16-bit Implementation (5 operations):

- 1) Shift immediate value left by 1 bit (halfword alignment)
- 2) Compute two’s complement negation (NOT + increment)
- 3) Check offset sign bit
- 4) Conditionally select addition or subtraction
- 5) Multiplex result based on sign

This manual approach does not trust the hardware’s signed arithmetic capabilities. Instead, it explicitly checks the sign, computes both forward and backward offsets, and selects between them. Each of these operations consumes additional LUTs

B. Synthesis Impact

FPGA synthesis tools (Vivado in this case) must implement each described operation in hardware. The 16-bit processor’s explicit step-by-step approach prevents certain optimizations

that would be possible with a cleaner implementation. Specifically:

- The synthesis tool cannot collapse the NOT-increment-add sequence into a single subtract operation
- The conditional multiplexing prevents datapath merging optimizations
- Multiple arithmetic units (adder AND subtractor) must be instantiated rather than sharing a single unit

In contrast, the 32-bit processor’s simple “shift and add” approach allows aggressive optimization. The synthesis tool recognizes this as standard signed addition and implements it efficiently using dedicated carry-chain resources and optimized adder structures.

C. Generalization to Other Logic

While branch calculation represents the primary inefficiency, the pattern suggests a broader principle: manual re-implementation of hardware-native operations typically wastes resources. Modern FPGAs and their synthesis tools are highly optimized for standard arithmetic, comparison, and logical operations. Attempting to “help” the tools by breaking operations into explicit sub-steps often prevents optimization and increases resource usage.

VI. DISCUSSION

A. Algorithmic Efficiency vs. Data Width

Our results demonstrate that algorithmic implementation choices can dominate architectural width considerations in determining hardware resource utilization. The 150% increase in branch calculation complexity (5 operations vs. 2) generated sufficient additional logic to not only eliminate the expected resource advantage of narrower data paths but actually reverse it.

This finding has important implications for processor design:

- **Efficiency is Multi-Dimensional:** The 16-bit processor wins on flip-flop usage and instantaneous power but loses on LUTs, execution time, and total energy. Designers must prioritize metrics based on application constraints.
- **Algorithm Matters More Than Width:** A well-designed 32-bit implementation can be more resource-efficient than a poorly-designed 16-bit implementation, challenging the assumption that narrower is always better.
- **Trust the Hardware:** Leveraging hardware-native operations (signed arithmetic, carry chains) typically produces more efficient results than manual implementation of the same operations.

B. Educational Implications

For computer architecture education, these findings suggest several lessons:

Teach Optimization Principles: Students should learn not just architectural concepts but also algorithmic optimization. The branch calculation example demonstrates how implementation choices affect hardware resource consumption.

Measure, Don’t Assume: Intuition about resource usage can be misleading. Synthesis results should be examined and understood rather than assumed based on high-level architectural features.

Code Quality Matters in Hardware: In software, poor algorithms affect performance but rarely affect memory footprint. In hardware, poor algorithms directly increase silicon area or FPGA resource usage.

C. Embedded Systems Design

For resource-constrained embedded systems, these results suggest that choosing a narrower architecture to save resources may backfire if the implementation is not carefully optimized. A 32-bit processor with clean, efficient logic may be preferable to a 16-bit processor with complex, inefficient implementations—even when resources are limited.

Additionally, the power vs. energy distinction becomes critical in battery-powered systems. While the 16-bit processor consumes less power per unit time, its slower execution means it must remain active longer, ultimately consuming more total energy and reducing battery life.

D. Limitations

Our study has several limitations that suggest directions for future work:

- We examined only single-cycle implementations; pipelined architectures may exhibit different resource-efficiency trade-offs
- Our analysis focused on one specific algorithmic inefficiency; other inefficiencies may exist that we did not fully characterize
- Results are specific to the Xilinx Artix-7 FPGA architecture; different FPGA families or ASIC implementations may show different patterns
- We tested with a single benchmark program; more comprehensive workload analysis would strengthen conclusions

VII. RELATED WORK

Extensive literature exists on MIPS processor implementations and comparative architecture studies. Patterson and Hennessy’s foundational work [1] established MIPS as an educational architecture and documented standard implementation approaches. Harris and Harris [2] provided detailed single-cycle and pipelined MIPS implementations commonly used in teaching.

Comparative studies of different bit-width architectures have primarily focused on performance trade-offs rather than resource efficiency paradoxes. Tang et al. [3] compared 32-bit and 64-bit RISC-V implementations on FPGAs, finding expected resource scaling with width. Our work differs in discovering and explaining a case where narrower architectures consume more resources due to algorithmic inefficiency.

Research on FPGA implementation optimization [4] has emphasized the importance of synthesis-friendly coding styles, but few studies have quantified the resource impact of specific

algorithmic choices in processor implementations. Our branch calculation analysis contributes a concrete example of how algorithmic decisions affect FPGA resource utilization.

VIII. CONCLUSION

This comparative analysis demonstrates that algorithmic efficiency can dominate architectural width considerations in determining hardware resource utilization and energy consumption. Our discovery that a 16-bit MIPS processor consumes 10% more FPGA LUTs than an equivalent 32-bit processor challenges conventional assumptions about resource scaling and highlights the critical importance of implementation quality.

Three key principles emerge from this work:

First, smart algorithms handling wide data can be more efficient than inefficient algorithms handling narrow data. The 32-bit processor's 2-operation branch calculation outperforms the 16-bit processor's 5-operation approach despite processing double the data width.

Second, instantaneous power consumption differs from total energy consumption. The 16-bit processor's 3.5% power advantage is overwhelmed by its 20% execution time penalty, resulting in 16% higher energy consumption overall.

Third, manual re-implementation of hardware-native operations typically wastes resources. Trusting synthesis tools to optimize standard arithmetic operations produces better results than explicit step-by-step implementations.

For educational processor implementations, these findings suggest teaching algorithmic optimization alongside architectural concepts. For embedded systems design, they demonstrate that careful implementation can matter more than architectural width in determining resource efficiency.

Future work should investigate additional sources of algorithmic inefficiency, extend the analysis to pipelined implementations, and examine whether similar patterns exist in other processor architectures. The methodology demonstrated here—discovery-driven comparative analysis with systematic root cause investigation—provides a template for identifying and understanding unexpected hardware efficiency results.

REFERENCES

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design MIPS Edition: The Hardware/Software Interface*, 5th ed. San Francisco, CA: Morgan Kaufmann, 2017.
- [2] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture*, 2nd ed. San Francisco, CA: Morgan Kaufmann, 2012.
- [3] Y. Tang, Z. Wang, and X. Zhou, "Comparative Analysis of 32-bit and 64-bit RISC-V Processors on FPGA," *IEEE Int. Conf. on Field-Programmable Technology*, pp. 156-163, 2018.
- [4] D. Koch, F. Hannig, and D. Ziener, "FPGAs for Software Programmers," in *Synthesis Lectures on Digital Circuits and Systems*, vol. 6, no. 2, pp. 1-122, 2011.
- [5] FPGA4Student, "Verilog code for 16-bit MIPS Processor," *fpga4student.com*, [Online]. Available: <https://www.fpga4student.com/2017/06/Verilog-code-for-16-bit-MIPS-Processor.html>
- [6] Xilinx, "Vivado Design Suite User Guide: Synthesis (UG901)," v2023.2, 2023.
- [7] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. San Francisco, CA: Morgan Kaufmann, 2019.

- [8] W. Wolf, *Computers as Components: Principles of Embedded Computing System Design*, 3rd ed. San Francisco, CA: Morgan Kaufmann, 2013.